06

Part 6 · Runtime Optimization & Plans

Let Spark re-plan at runtime, and learn to read what it decided.

- Adaptive Query Execution
- Partition coalescing
- Adaptive joins
- Config tuning
- explain()
- Statistics & CBO

6.1 Adaptive Query Execution (AQE)

★★★

IN ONE LINE

Spark revises the query plan at runtime using real statistics after each shuffle stage.

Adaptive Query Execution (AQE) lets Spark change the query plan **while the query runs**. After each shuffle stage finishes, Spark knows the real sizes of the data and can fix wrong estimates made during planning. It is the single most useful automatic optimisation in modern Spark.

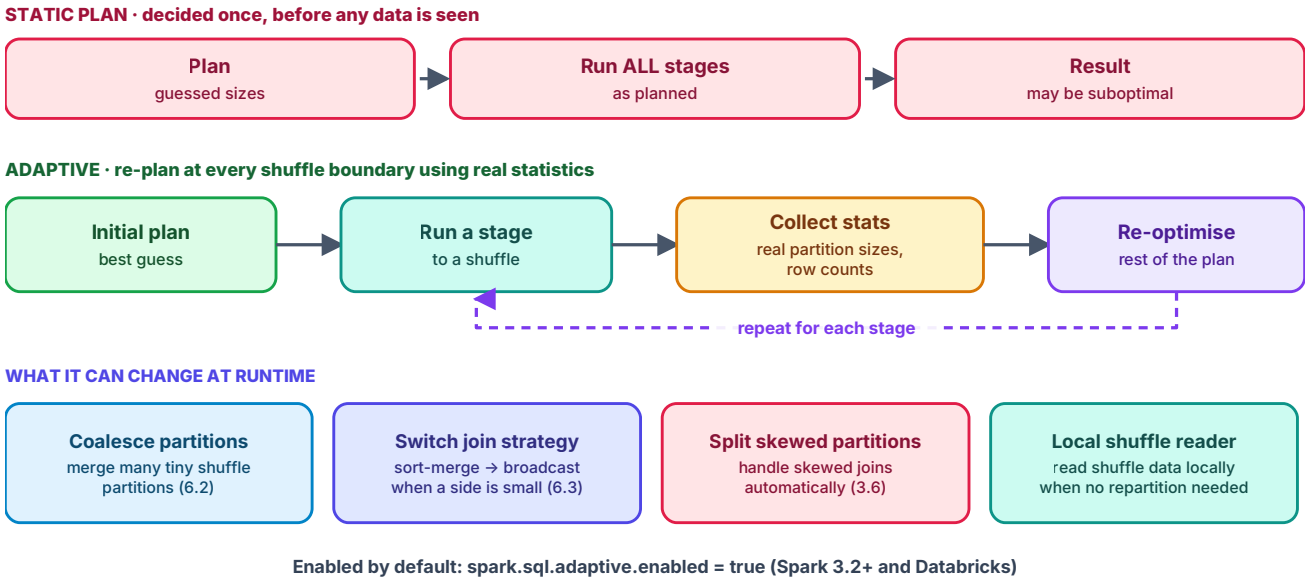


Figure 6.1 — AQE re-optimises the remaining plan using runtime statistics

Feature	Switch	Default
AQE overall	`spark.sql.adaptive.enabled`	true
Coalesce shuffle partitions	`spark.sql.adaptive.coalescePartitions.enabled`	true
Skew join optimisation	`spark.sql.adaptive.skewJoin.enabled`	true
Local shuffle reader	`spark.sql.adaptive.localShuffleReader.enabled`	true

REMEMBER

AQE only works on queries that contain a **shuffle or broadcast exchange** (otherwise there is nothing to re-plan), and it cannot fix problems that lie before the first shuffle (e.g. too many input files).

INTERVIEW ANGLE

In `explain()` an AQE query starts with **AdaptiveSparkPlan isFinalPlan=false** before running; the Spark UI SQL tab shows the **final** plan after execution. Compare them to see what AQE changed.

6.2 Adaptive Partition Coalescing

★★★

IN ONE LINE

AQE merges small shuffle partitions at runtime so tasks are neither tiny nor too many.

With the default `shuffle.partitions = 200`, a small query still creates 200 tiny shuffle partitions. **Adaptive partition coalescing** watches the real shuffle output sizes and **merges neighbouring small partitions** toward a target size, so you get fewer, bigger tasks — automatically.

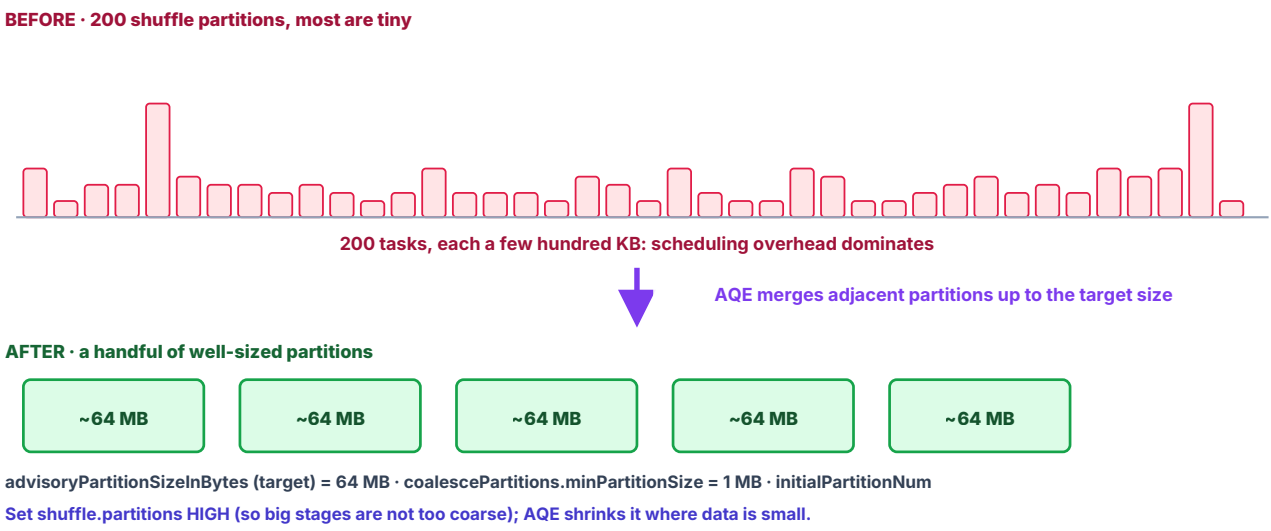


Figure 6.2 — Many tiny shuffle partitions merged to the advisory size

Setting	Default	Role
`spark.sql.adaptive.coalescePartitions.enabled`	true	Turns coalescing on
`spark.sql.adaptive.advisoryPartitionSizeInBytes`	64 MB	Target size after coalescing
`spark.sql.adaptive.coalescePartitions.minPartitionSize`	1 MB	Never produce partitions smaller than this
`spark.sql.adaptive.coalescePartitions.initialPartitionNum`	(shuffle.partitions)	Starting count before merging

Benefits

- No hand-tuning of shuffle partitions per query.
- Fewer tasks and fewer small output files.
- Works with changing data volumes day to day.

Limits

- Cannot **increase** partitions or fix skew by itself.
- Not applied after `repartition(n)` with an explicit count.
- Streaming queries do not use AQE.

INTERVIEW ANGLE

Interview line: “I set shuffle partitions high and let AQE coalesce; if a stage still shows spill or huge tasks I raise it or investigate skew.”

6.3 Adaptive Join Optimization

★★★

IN ONE LINE

AQE can switch join strategies at runtime when actual data sizes differ from estimates.

Estimates made before running are often wrong: a filter may shrink a table from 2 TB to 6 MB, but Spark planned a sort-merge join because it could not know. **Adaptive join optimisation** fixes this at runtime by **changing the join strategy** once real sizes are known.

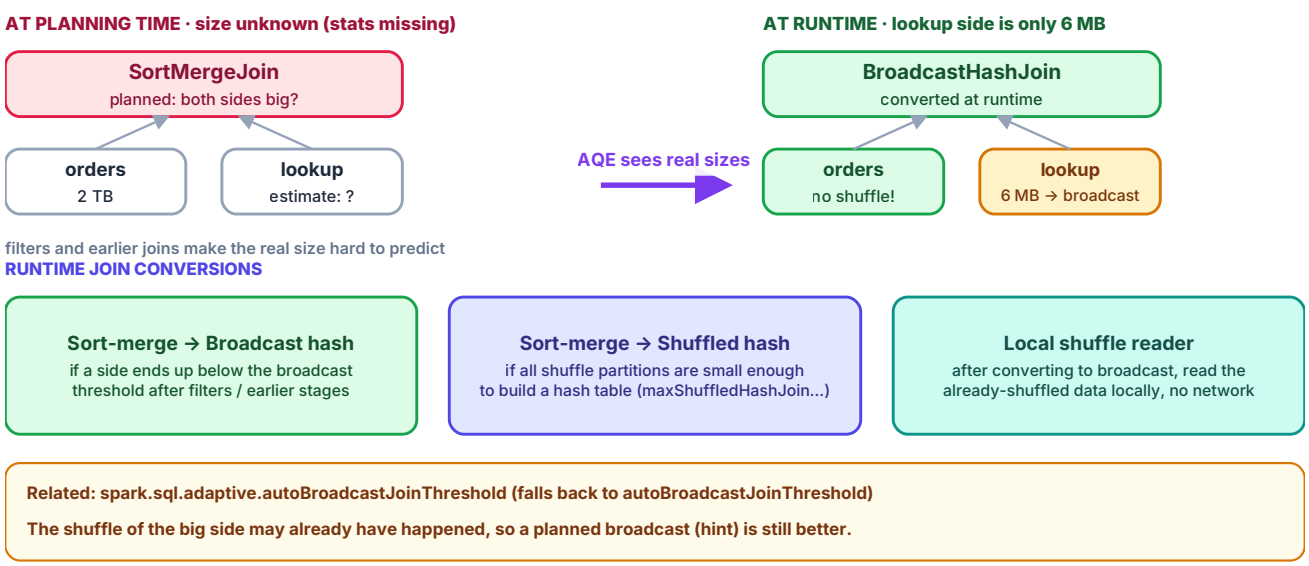


Figure 6.3 — AQE converts a planned sort-merge join into a broadcast join

	Static planning	AQE
Based on	Table statistics and guesses	Actual sizes of completed shuffle stages
Wrong estimate	Stuck with a poor join	Plan corrected mid-query
Cost	None	Small overhead; shuffle of the big side may already be done

INTERVIEW ANGLE

If you *know* a side is small, a **broadcast hint** still wins because it avoids the big-side shuffle completely. Use AQE as the safety net for cases you cannot predict.

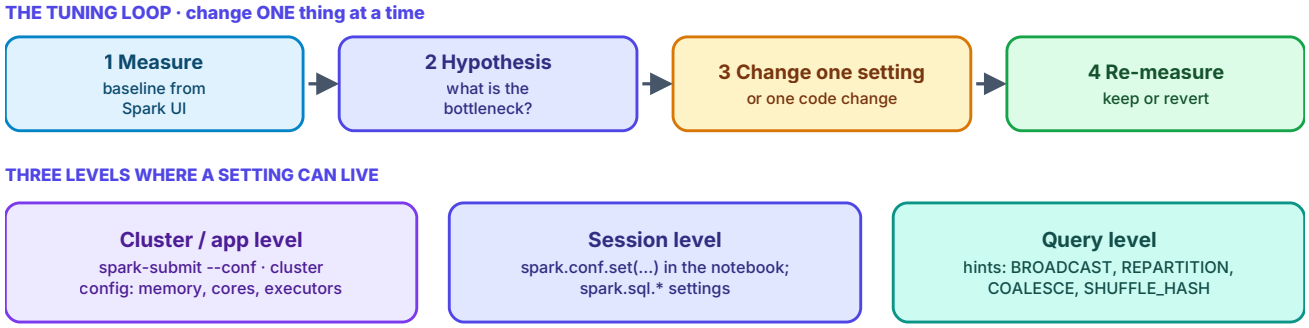
6.4 Spark Configuration Tuning

★★★

IN ONE LINE

A handful of spark.sql.* and resource settings explain most gains; change one at a time and measure.

Spark has hundreds of settings, but a few explain most real-world wins. The golden rule: **measure first, change one thing, measure again**. Random tweaking wastes time and can make things worse.



Setting (spark.sql.*)	Default	Tune when
shuffle.partitions	200	shuffles too big / too small (AQE helps)
autoBroadcastJoinThreshold	10 MB	dimension tables a bit larger than 10 MB
files.maxPartitionBytes	128 MB	too few / too many read tasks
adaptive.enabled	true	only to compare / disable for debugging
adaptive.advisoryPartitionSizeInBytes	64 MB	larger / smaller post-shuffle tasks

Figure 6.4 — Tuning loop, setting levels and the key defaults

Symptom	Likely setting / action
Spill, big shuffle tasks	Raise `spark.sql.shuffle.partitions` (or rely on AQE); give executors more memory per core
Tiny tasks, long scheduling	Lower shuffle partitions; larger `files.maxPartitionBytes`; compact small files
Join shuffles a small table	Raise `autoBroadcastJoinThreshold` or use a broadcast hint
Driver OOM on collect / broadcast	Increase driver memory only after reducing what goes to the driver
Container killed (PySpark)	Raise `spark.executor.memoryOverhead`
Cannot reproduce speed-up	Check that AQE and caching state are the same in both runs

INSPECTING AND SETTING CONFIGS

```
spark.conf.get('spark.sql.shuffle.partitions')     # inspect current value
spark.conf.set('spark.sql.shuffle.partitions', '400')   # set for this session
spark.sql('SET spark.sql.autoBroadcastJoinThreshold')    # SQL equivalent
spark.sparkContext.getConf().getAll()                    # all settings
```

COMMON TRAP

Some settings are **static** and must be set when the application starts (executor memory, cores); you cannot change them with `spark.conf.set` mid-session. On Databricks, set them in the cluster's Spark config.

6.5 Inspecting Optimized Plans with explain()

★★★

IN ONE LINE

Read the physical plan to verify joins, shuffles, pruning and pushdown.

`explain()` prints the plan Spark chose. It is how you **verify** that an optimisation really happened instead of assuming it did.

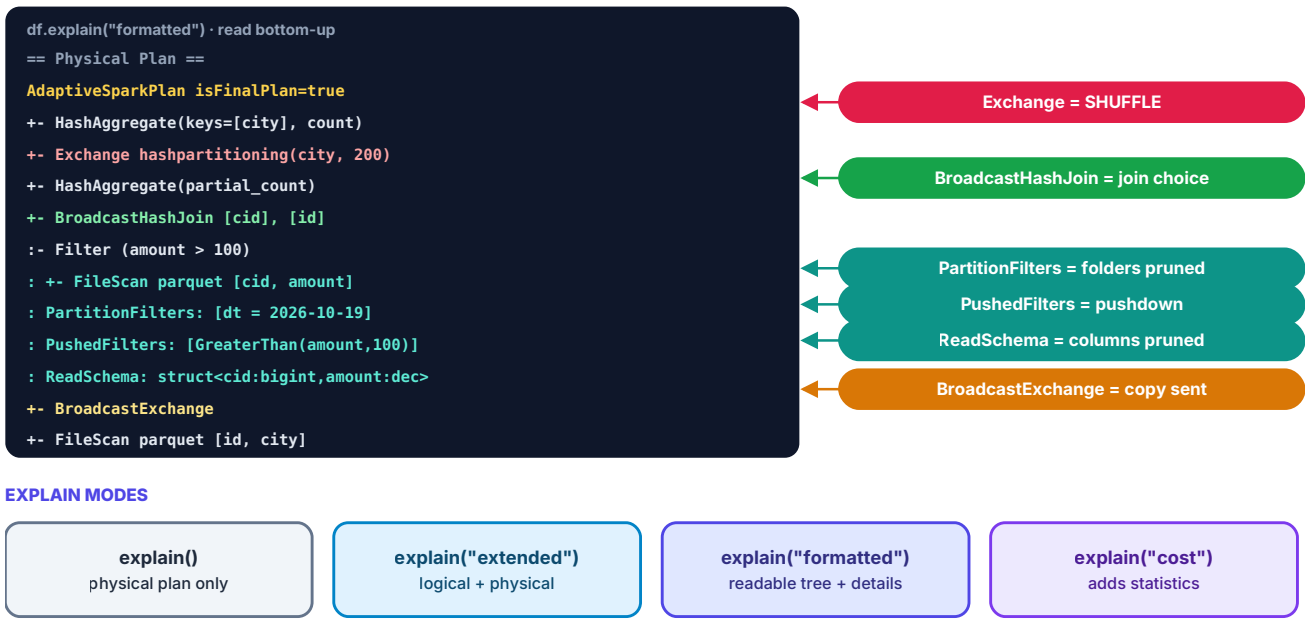


Figure 6.5 — An annotated physical plan

Look for	Meaning	Good / bad
Exchange	A shuffle	Count them; fewer is better
BroadcastHashJoin vs **SortMergeJoin**	The join algorithm	Broadcast for small dimensions
PartitionFilters	Partition pruning applied	Empty on a partitioned table = full scan
PushedFilters	Filters handed to the reader	Missing = filter not pushable (UDF, cast)
ReadSchema	Columns actually read	All columns = no pruning (SELECT *)
InMemoryTableScan	Reading from cache	Confirms the cache is used
CartesianProduct / BroadcastNestedLoopJoin	No equi-join key	Almost always a bug

INTERVIEW ANGLE

A great interview habit: after describing any optimisation, add *"and I would confirm with **explain()** / the SQL tab that the join became a broadcast join, that PushedFilters appear, and that the number of Exchanges dropped."*

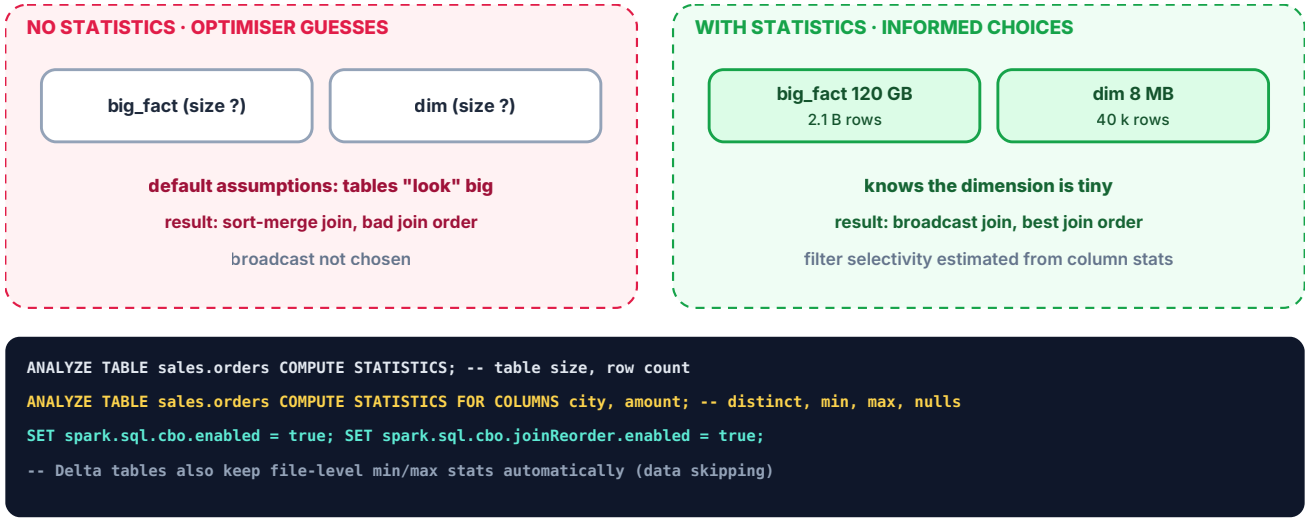
6.6 Statistics and the Cost-Based Optimizer

★★☆

IN ONE LINE

Table and column statistics let Spark estimate sizes and choose better join strategies and orders.

Catalyst has two kinds of rules: **rule-based** (always beneficial, like predicate pushdown) and **cost-based** (choices that need data size estimates, like join order or whether to broadcast). The cost-based part is only as good as the **table and column statistics** it can see.




```
ANALYZE TABLE sales.orders COMPUTE STATISTICS; -- table size, row count
ANALYZE TABLE sales.orders COMPUTE STATISTICS FOR COLUMNS city, amount; -- distinct, min, max, nulls
SET spark.sql.cbo.enabled = true; SET spark.sql.cbo.joinReorder.enabled = true;
-- Delta tables also keep file-level min/max stats automatically (data skipping)
```

Figure 6.6 — Statistics let the optimiser choose better joins

Statistic	Collected by	Used for
Table size, row count	`ANALYZE TABLE … COMPUTE STATISTICS`	Broadcast decisions, join order
Column stats (distinct, min, max, nulls)	`… FOR COLUMNS col1, col2`	Filter selectivity, join cardinality
File-level min/max (Delta)	Automatically on write	Data skipping (5.7)
Runtime stats	AQE after each shuffle	Re-planning (6.1 – 6.3)

COMMON TRAP

The cost-based optimiser (`spark.sql.cbo.enabled`) is **off by default in open-source Spark**, and stale statistics are worse than none. Managed platforms may enable or maintain statistics for you — check your environment, and refresh statistics after big data changes.

INTERVIEW ANGLE

Because AQE uses **runtime** statistics, it covers many cases where stats are missing. Still, for stable, repeatedly queried tables, up-to-date statistics improve plans from the very first stage.